



<Coding[🎵]onata />

3 Types of Middleware Creation in ASP.NET Core



Inline Middleware

Inline Middleware

Quickly define middleware **directly in the pipeline** using a lambda expression.

Best for simple logic that **doesn't need to be reused** elsewhere.

Easy to add for common use cases like logging, timing, or minor headers.

Inline Middleware

```
app.Use(async (context, next) =>
{
    var logger = context
        .RequestServices
        .GetRequiredService<ILogger<Program>>();
    logger.LogInformation(
        "Inline Middleware: Handling request {Path}",
        context.Request.Path
    );

    await next();

    logger.LogInformation(
        "Inline Middleware: Finished handling request"
    );
});
```

2

Convention- based Middleware

Convention-based Middleware

Encapsulates logic in a class, making it reusable and testable.

Follows the "convention" of creating a Middleware class.

Ideal when the middleware needs configuration or will be reused across multiple projects.

Convention-based Middleware

```
public class LoggingMiddleware(
    RequestDelegate next,
    ILogger<LoggingMiddleware> logger
)
{
    public async Task InvokeAsync(HttpContext context)
    {
        logger.LogInformation(
            "Convention-based Middleware: Handling request {Path}",
            context.Request.Path
        );

        await next(context);

        logger.LogInformation(
            "Convention-based Middleware: Finished handling request"
        );
    }
}

// Inside Program.cs

// Register in DI
builder.Services.AddTransient<LoggingMiddleware>();

// Use in Pipeline
app.UseMiddleware<LoggingMiddleware>();
```

3

Factory- based Middleware

Factory-based Middleware

Implements `IMiddleware` interface instead of taking `RequestDelegate` via constructor.

ASP.NET Core handles middleware instantiation using dependency injection.

Ideal when the middleware has dependencies (e.g., services, configuration, logging, etc.).

Factory-based Middleware

```
public class LoggingFactoryMiddleware(
    ILogger<LoggingMiddleware> logger
)
    : IMiddleware
{
    public async Task InvokeAsync(
        HttpContext context,
        RequestDelegate next)
    {
        logger.LogInformation(
            "IMiddleware: Handling request {Path}",
            context.Request.Path
        );

        await next(context);

        logger.LogInformation(
            "IMiddleware: Finished handling request"
        );
    }
}

// Register in DI
builder.Services.AddTransient<LoggingFactoryMiddleware>();

// Use in Pipeline
app.UseMiddleware<LoggingFactoryMiddleware>();
```

Key Notes

Use **convention-based** middleware when:

- You don't need advanced DI features.
- The middleware is stateless or only needs logging/configuration.
- You're creating quick reusable middleware across projects.

Key Notes

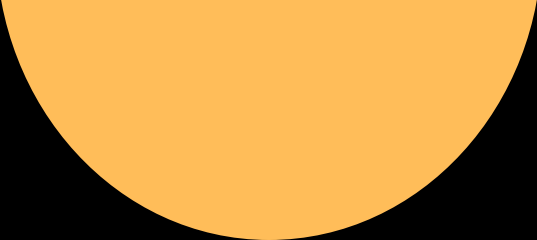
Use **factory-based** middleware when:

- You need scoped services (e.g., DbContext, user context).
- You want full lifetime control (transient/singleton).
- You're writing testable, decoupled, and clean middleware components.
- You prefer a fully DI-managed environment.

Found this useful?



Consider Reposting



Thank You

Follow me for more content



Aram Tchekrekjian



Get Free tips and Tutorials in .NET and C#



Join 800+ Readers

CodingSonata.com/newsletters

<CodingSonata />